

Universidad de Valladolid

Práctica 2: Pruebas de carga, Locust y Prometheus

Grado en Ingeniería Informática
Evaluación de Sistemas Informáticos

Grupo 16

Índice

1. Introducción **3**
1.1. Descripción del sistema bajo prueba . . . 3

2. Estructura del proyecto **3**

3. Prometheus **4**
3.1. Instalación . . . 4

4. Métricas que vamos a emplear en la práctica **5**
4.1. Sintaxis de los contadores . . . 6
4.2. Deciciones importantes sobre *metrics.py* . . . 10

5. Modificaciones a *models.py* **10**
5.1. store_authorization_code . . . 10
5.2. consume_authorization_code . . . 11
5.3. issue_token_pair . . . 11
5.4. validate_refresh_token . . . 11
5.5. revoke_refresh_token . . . 12
5.6. Exposición del endpoint /metrics — app.py . . . 12

6. Modificaciones a *app.py* **15**
6.1. Exposición del endpoint /metrics . . . 15
6.2. Registro automático de métricas HTTP . . . 15
6.3. Instrumentación de flujos OAuth2 — oauth2_routes.py . . . 16

7. Locust **18**
7.1. Instalación . . . 18
7.2. Explicación del diseño del locustfile . . . 38
7.3. Configuración de las pruebas . . . 38
7.4. Protección de la VM durante las pruebas . . . 39
7.5. Comportamientos del sistema frente a los dos escenarios de prueba . . . 39
7.5.1. Condiciones normales . . . 40
7.5.2. Condiciones de estrés . . . 40
7.5.3. Flujo de ejecición de las aplicaciones . . . 41

8. Verificación manual de métricas EDR **41**

9. Enfoque EDR del sistema de métricas **42**
9.1. Métricas adicionales implementadas . . . 42
9.1.1. Rendimiento del sistema . . . 42
9.1.2. Estado de tokens . . . 43
9.1.3. Detección de ataques . . . 43

10. Paneles de grafana **43**
10.1. Ataques de fuerza burta vs *credential stuffing* . . . 45
10.2. Ratio de Tokens . . . 45
10.3. Fallos del login de los usuarios . . . 46
10.4. Fallos del login de los usuarios . . . 46
10.5. Logins correctos en el sistema . . . 47

10.6. Peticiones a los diferentes endpoints de nuestro sistema 47
10.7. Distribución de las peticiones en nuestro sistema 48
10.8. Ratio peticiones correctas fallidas por los usuarios 48
10.8.1. Tiempo de respuesta a cada *endpint* 49
10.8.2. Tipos de *tokens* en nuestro sistema. 49
10.8.3. Distribución de *tokens* en el sistema 50
10.8.4. Intentos de ataque de fuerza bruta 50
10.8.5. Flujos de autenticació en cada uno de los *endpoints* 51
10.9. Intento de login por parte de usuarios desconocidos 52

1 Introducción

Para esta práctica estaremos empleando un servidor de autenticación *OAuth2*, el cual es el servidor por excelencia para hacer de mediación en la tarea de distribución de claves de acceso sobre el que realizaremos pruebas de carga con Locust, el cual es un sistema de monitorización que aplica el modelo *pull*, el cual, está apuntando a nuestro *endpoint* y cada 15 segundos (según lo definido en su *.yml*) anota todo lo que está pasando, estos eventos serán visualizados posteriomente en Grafana.
De tal manera que tenemos 3 objetivos:

1. Instrumentación del backend con métricas Prometheus (haremos incapie en esta tecnología en 3).
2. Despliegue y configuración de Prometheus y Grafana.
3. Pruebas de carga y estrés con Locust.

1.1 Descripción del sistema bajo prueba

El backend proporcionado es un servidor de autenticación OAuth2 desarrollado en Python con Flask y SQLite, que implementa los tres flujos principales definidos en el RFC 6749:

- **Authorization Code**: flujo orientado a aplicaciones web donde un usuario humano se autentica mediante un formulario de login. El servidor emite un código temporal que posteriormente se canjea por un par de tokens.
- **Client Credentials**: flujo orientado a aplicaciones máquina a máquina donde no hay un usuario humano. La aplicación se autentica directamente con sus credenciales y obtiene un token de acceso.
- **Refresh Token**: flujo que permite renovar un token de acceso caducado sin necesidad de repetir el proceso de autenticación completo.

La aplicación se gestiona con `uv`, el cual ya empleamos en nuestra primera práctica y como se explicó en el informe es un gestor de paquetes para Python más moderno que el típico pip, y se ejecuta en el puerto 5001.

2 Estructura del proyecto

Para entender el proyecto vamos a ver qué hacen los ficheros que han sido proporcionados por el profesor y los que hemos tenido que crear para el mismo:

Fichero	Descripción
`app.py`	Punto de entrada de la aplicación Flask. Registra blueprints, configura la base de datos y expone el endpoint `/metrics`.
`oauth2_routes.py`	Implementación de los endpoints OAuth2: `/authorize` y `/token`.
`admin_routes.py`	Endpoints de administración bajo `/admin/clients` para gestionar clientes OAuth2.
`models.py`	Capa de acceso a datos: clientes, códigos de autorización y tokens.
`database.py`	Gestión de la conexión SQLite y definición del esquema.
`metrics.py`	Definición centralizada de todas las métricas Prometheus (fichero creado en esta práctica).
`seed.py`	Script de inicialización que crea dos clientes de prueba en la base de datos.

Cuadro 1: Estructura de ficheros modificados en el proyecto

3 Prometheus

Prometheus es una base de datos de series temporales de tipo *pull*, esto quiere decir que es la propia base de datos quien periódicamente consulta a las aplicaciones por sus métricas, de tal manera que las aplicaciones toman un papel pasivo, donde solo se encagan de realizar sus tareas y sólo tiene que exponer un *enpoint* llamado */metrics* con los datos en el mismo formato que emplea esta tecnología.

3.1 Instalación

Descargamos el binario de github y lo dejamos descomprimido en la VM

```
wget https://github.com/prometheus/prometheus/releases/
   download/\
v3.3.1/prometheus-3.3.1.linux-amd64.tar.gz
tar -xvf prometheus-3.3.1.linux-amd64.tar.gz
cd prometheus-3.3.1.linux-amd64
```

```
uv add prometheus-client
```

Aquí uv juega un papel fundamental, puesto que de *add* de manera automática instala la librería, actualiza el .toml del proeycto y registra la dependencia en el *uv.lock*.
Para ello hemos tenido que modificar el fichero de prometheus de la siguiente manera.

```
global:
  #intervalo de busqueda de las metricas en el localhost 5001
  scrape_interval: 15s
```

```
  #intervalo de evaluacion de las reglas, para que nos llame grafana si se
     lia
  #no lo vamos a emplear en la practica
  evaluation_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  #Esto es un nuevo trabajo que le dice al prometheus donde tiene que ir a
     buscar
  - job_name: "oauth2"
    static_configs:
      - targets: ["localhost:5001"]
```

En este fichero lo que estamos haciendo es definir el comportamiento del monitoreo, en el cual estableceos que recoja méticas cada 15 segundos, del la aplicaicón que está corriendo en el 5001 y del enpoint */metrics*.
Ahora que lo tenemos todo configurado, para ejecutar el prometheus en nuestro sistema simplemente ejecutamos el siguiente comando especificando que su fichero de configuración es es el que hemos hecho antes.

```
./prometheus --config.file=prometheus.yml
```

4 Métricas que vamos a emplear en la práctica

Emplearemos 2 de las 4 métricas que nos proporciona la apliación, puesto que son las que más se adaptan al planteamiento que tenemos para la práctica:

- **Counter**: contador de incremento monotónico. Solo puede crecer. Se usa para contar eventos como peticiones recibidas, tokens emitidos o errores producidos. Por convención, los counters tienen el sufijo `_total`.
- **Gauge**: contador que puede crecer y decrecer. Se usa para valores que fluctúan como memoria usada o conexiones activas.

Definición de las métricas que vamos a recoger

Hemos creado un fichero *metrics.py* en el que hemos añadido todos los contadores, con dos objetivos, por una parte centralizarlo todo en un fichero, facilitando el desarrollo y la depuración del código y por otra parte, evitar las importaciones circulares de módulos.
Este fenómeno ocurre cuando dos ficheros se importan entre ellos, es decir, si por ejemplo en *oauth2_metrics.py* importamos cosas de *app.py* y en el último importamos fucniones del primero, esto lo solucionamos gestionando todos los contadores en un mismo fichero.

4.1 Sintaxis de los contadores

Antes de mostrar todos los contadores, vamos a explicar un poco la sintaxis de este fichero, puesto que es una tecnología nueva, que al menos en la carrera no hemos tocado mucho:

- auth_flows_started = Counter(: nombre de variable la cual es un contador, la cual sólo puede incrementarse
- .ºauth2_auth_flows_started_total": nombre de la métrica para su posterior integración con grafana
- "Flujos de autenticacion iniciados", : descripción de la metrica
- ["*grant_type*"]: la etiqueta para luego poder filtrarlo en el grafana.)

```python
#Codigo para recopilar las metricas de Prometheus.

from prometheus_client import (
    Counter,
    Histogram,
    Gauge
)

#Flujos de autenticacion
# Contabiliza los intentos de llamada al endpoint de authorized o de token (exitosos o no)
auth_flows_started = Counter(
    "oauth2_auth_flows_started_total",
    "Flujos de autenticación iniciados",
    ["grant_type"] #permite almacenar el resultado de cada contador de manera
        independiente para poder tener una cifra de cada uno.
)

#Contabiliza los intentos exitosos de autenticacion en el sistema
auth_flows_success = Counter(
    "oauth2_auth_flows_success_total",
    "Flujos de autenticación completados con éxito",
    ["grant_type"]
)

#Contabiliza los intentos fallidos de autenticacion en el sistem
auth_flows_failed = Counter(
    "oauth2_auth_flows_failed_total",
```

```
    "Flujos de autenticación fallidos",
    ["grant_type"]
)

#Tokens emitidos por la aplicacion a nivel general
tokens_issued = Counter(
    "oauth2_tokens_issued_total",
    "Tokens emitidos",
    ["token_type", "grant_type"]
)

#Tokens emitidos (separados por tipo)
access_tokens_issued = Counter(
    "oauth2_access_tokens_issued_total",
    "Access tokens emitidos",
    ["grant_type"]
)
refresh_tokens_issued = Counter(
    "oauth2_refresh_tokens_issued_total",
    "Refresh tokens emitidos",
    ["grant_type"]
)

#Llamadas HTTP para medir el numero de peticiones a nivel general
http_requests = Counter(
    "oauth2_http_requests_total",
    "Número de llamadas HTTP",
    ["method", "endpoint", "status_code"]
)

#Llamadas HTTP (separadas por endpoint)
http_requests_token = Counter(
    "oauth2_http_requests_token_total",
    "Llamadas al endpoint /token",
    ["method", "status_code"]
)

http_requests_authorize = Counter(
    "oauth2_http_requests_authorize_total",
    "Llamadas al endpoint /authorize",
    ["method", "status_code"]
)

http_requests_admin = Counter(
    "oauth2_http_requests_admin_total",
    "Llamadas a los endpoints /admin",
    ["method", "status_code"]
)

#Permite la medicion del numero de request que estan en proceso en este momento
requests_in_progress = Gauge(
    "oauth2_requests_in_progress",
    "Peticiones HTTP en curso en este momento",
```

```
    ["endpoint"]
)

#Metricas extra para el planteamiento de ser un EDR
#Con esto podemos medir el tiempo medio de duracion de un request
request_duration = Histogram(
    "oauth2_request_duration_seconds",
    "Tiempo de respuesta de cada peticion",
    ["endpoint", "method"],
    # 0.25 es el umbral de 250ms que marca el enunciado como referencia
    buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
)

#Con esto intentamos ver si esta pasando algo raro con los tokens
#Info sobre los tokens activos en el sistema
active_tokens = Gauge(
    "oauth2_active_tokens",
    "Tokens de acceso activos en el sistema",
    ["token_type"]
)

#Info sobre los tokens caducados
tokens_revoked = Counter(
    "oauth2_tokens_revoked_total",
    "Tokens revocados",
    ["token_type"]
)

#Nos da inormacion sobre los codigos emitos pero no empleados
pending_auth_codes = Gauge(
    "oauth2_pending_auth_codes",
    "Codigos de autorizacion emitidos pero no canjeados"
)

# Intentos de uso de tokens ya revocados
# Señal de posible token replay attack
revoked_token_usage = Counter(
    "oauth2_revoked_token_usage_total",
    "Intentos de uso de tokens ya revocados"
)

# Intentos de uso de tokens expirados
expired_token_usage = Counter(
    "oauth2_expired_token_usage_total",
    "Intentos de uso de tokens expirados"
)

# Intentos de reutilizacion de codigos de autorizacion ya usados
# Señal de posible authorization code replay attack
auth_code_reuse = Counter(
    "oauth2_auth_code_reuse_total",
    "Intentos de reutilizacion de codigos ya usados"
)
```

```
133
134 # Intentos fallidos de autenticacion por client_id
135 # Util para detectar ataques de fuerza bruta contra un cliente concreto
136 brute_force_attempts = Counter(
137     "oauth2_brute_force_attempts_total",
138     "Intentos fallidos de autenticacion por cliente",
139     ["client_id"]
140 )
141
142 # Intentos fallidos de login de usuario
143 # Util para detectar ataques de fuerza bruta contra usuarios concretos
144 user_auth_failed = Counter(
145     "oauth2_user_auth_failed_total",
146     "Intentos fallidos de autenticacion de usuario",
147     ["username", "client_id"]
148 )
149
150 # Operaciones de gestion de clientes OAuth2
151 clients_created = Counter(
152     "oauth2_admin_clients_created_total",
153     "Clientes OAuth2 creados via API admin"
154 )
155
156 clients_deleted = Counter(
157     "oauth2_admin_clients_deleted_total",
158     "Clientes OAuth2 eliminados via API admin"
159 )
160
161 clients_listed = Counter(
162     "oauth2_admin_clients_listed_total",
163     "Consultas al listado de clientes"
164 )
165
166 user_auth_success = Counter(
167     "oauth2_user_auth_success_total",
168     "Inicios de sesion exitosos por usuario",
169     ["username"]
170 )
171
172 #Vamos a simular el comportamiento de un usuarios
173 #metrica para los clicks del formulario.
174 login_page_visits = Counter(
175     "oauth2_login_page_visits_total",
176     "Visitas a la pagina de login por client_id",
177     ["client_id"]
178 )
179 #Intentos de login con un usario desconocido
180 unknown_client_attempts = Counter(
181     "oauth2_unknown_client_attempts_total",
182     "Intentos de acceso con client_id desconocido"
183 )
184
185 # Un valor alto indica credential stuffing
```

```
# Un valor bajo con muchos fallos indica fuerza bruta clasica
unique_failed_usernames = Gauge(
    "oauth2_unique_failed_usernames_active",
    "Numero de usernames distintos con fallos de login activos"
)
```

Las **etiquetas** (*labels*) permiten que un mismo counter almacene valores independientes para cada combinación de etiquetas. Por ejemplo, `auth_flows_started` con la etiqueta `grant_type` almacena un contador separado para cada tipo de flujo:

```
oauth2_auth_flows_started_total{grant_type="
   client_credentials"} 10.0
oauth2_auth_flows_started_total{grant_type="
   authorization_code"} 3.0
oauth2_auth_flows_started_total{grant_type="refresh_token"}
    5.0
```

4.2 Deciciones importantes sobre *metrics.py*

Una decisión de diseño importante fue no añadir `client_id` como etiqueta en `tokens_issued`. Aunque inicialmente se contempló para tener información por cliente, se descartó porque Prometheus crea una serie temporal por cada combinación única de etiquetas. Con una etiqueta de alta cardinalidad como `client_id` el número de series crecería proporcionalmente al número de clientes, lo que en una VM con 2GB de RAM podría causar problemas de memoria durante las pruebas de carga.

5 Modificaciones a *models.py*

textttmodels.py es la capa de acceso a datos. Se ha instrumentado con métricas en cuatro funciones:

5.1 store_authorization_code

Se incrementa `pending_auth_codes` cada vez que se genera un código de autorización, reflejando que existe un código emitido pero no canjeado:

```
        def store_authorization_code(...) -> str:
        # ... codigo original ...
        db.commit()
        # A new code has been issued but not yet consumed
        pending_auth_codes.inc()
        return code
```

Listing 1: Instrumentacion de store_authorization_code

5.2 consume_authorization_code

Se decrementa `pending_auth_codes` cuando el código se consume correctamente. Además se detectan dos casos de seguridad relevantes: el intento de reutilización de un código ya usado (`auth_code_reuse`) y el intento de uso de un código expirado:

```
        def consume_authorization_code(code: str) -> Optional[dict]:
        # ... codigo original ...
        if data["used"]:
        # Someone is trying to reuse an already consumed code
        # This is a sign of an authorization code replay attack
        auth_code_reuse.inc()
        return None

        if time.time() > data["expires_at"]:
        return None

        db.execute("UPDATE authorization_codes SET used = 1 ...")
        db.commit()
        # Code consumed correctly, no longer pending
        pending_auth_codes.dec()
        return data
```

Listing 2: Instrumentacion de consume_authorization_code

5.3 issue_token_pair

Se incrementan los gauges de tokens activos cada vez que se emiten tokens. Se distingue entre access tokens y refresh tokens:

```
        def issue_token_pair(...) -> dict:
        # ... codigo original ...
        db.commit()
        # Track active tokens in the system
        active_tokens.labels(token_type="access").inc()
        if refresh_token:
        active_tokens.labels(token_type="refresh").inc()
        return response
```

Listing 3: Instrumentacion de issue_token_pair

5.4 validate_refresh_token

Se detectan los dos casos de fallo relevantes para la seguridad: el intento de uso de un token revocado y el intento de uso de un token expirado. Es importante que estos contadores se incrementen aquí en `models.py` y no en `oauth2_routes.py`, porque esta función tiene acceso directo al motivo exacto del fallo:

```
        def validate_refresh_token(token: str) -> Optional[dict]:
        # ... codigo original ...
        if data["revoked"]:
        # Someone is using a token after it has been revoked
```

```
        # This is a sign of a token replay attack
        revoked_token_usage.inc()
        return None

        if time.time() > data["expires_at"]:
        # Someone is trying to use an expired token
        expired_token_usage.inc()
        return None

        return data
```

Listing 4: Instrumentacion de validate_refresh_token

5.5 revoke_refresh_token

Se incrementan los counters de tokens revocados y se decrementan los gauges de tokens activos. La rotación de tokens del flujo `refresh_token` revoca tanto el refresh token antiguo como su access token asociado:

```
        def revoke_refresh_token(token: str) -> None:
        # ... codigo original ...
        db.commit()
        # Both the refresh token and its associated access token are revoked
        tokens_revoked.labels(token_type="refresh").inc()
        tokens_revoked.labels(token_type="access").inc()
        # Decrement active tokens accordingly
        active_tokens.labels(token_type="refresh").dec()
        active_tokens.labels(token_type="access").dec()
```

Listing 5: Instrumentacion de revoke_refresh_token

5.6 Exposición del endpoint /metrics — app.py

Se modifica `app.py` para exponer el endpoint `/metrics` y registrar automáticamente todas las peticiones HTTP mediante decoradores Flask:

```
"""
app.py --Flask application entry point.

Wires together the database, blueprints, and configuration.
Run with: uv run python app.py
"""
#DISCLAIMER we've commented in english to mantain the consistence of the code

#request had the request info
#g will be use to store the request live time
from flask import Flask, request, g
#wsi will be the way promethus get their data
from prometheus_client import make_wsgi_app
#use for the comunication btw the wsgi and metrics.py
```

```python
from werkzeug.middleware.dispatcher import DispatcherMiddleware
import time

from database import close_db, init_db
from oauth2_routes import oauth2
from admin_routes import admin
#we need this lib for the metrics that we've allready defined on metrics.py
from metrics import (
       http_requests,
       http_requests_token,
       http_requests_admin,
       http_requests_authorize,
       request_duration,
       requests_in_progress
)

from werkzeug.serving import run_simple
# ---------------------------------------------------------------------------
# Hardcoded test users (resource owners for the authorization code flow).
# In production these would live in a proper user store.
# ---------------------------------------------------------------------------
USERS = {
    "testuser": "testpass",
    "admin": "adminpass",
}


def create_app() -> Flask:
    app = Flask(__name__)
    app.secret_key = "dev-secret-key-change-in-production"

    # Store users in app config so routes can access them
    app.config["USERS"] = USERS

    # Register blueprints
    app.register_blueprint(oauth2)
    app.register_blueprint(admin)

    # Database lifecycle
    app.teardown_appcontext(close_db)

    # Ensure tables exist on first launch
    with app.app_context():
        init_db()

    #Automatic HTTP metrics log
    # the Flask decorator
    @app.before_request
    def before_request():
        # Store the moment of the request arrives
        g.start_time = time.time()
        requests_in_progress.labels(endpoint=request.path).inc()
    #This force that every request must execute this methods
```

```
    @app.after_request
    def after_request(response):
        # get the request endpoint
        endpoint = request.path

        # we increment the general counter
        http_requests.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=str(response.status_code)
        ).inc()

        # We increment the specific counter for each endpoint
        if "/token" in endpoint:
            http_requests_token.labels(
                method=request.method,
                status_code=str(response.status_code)
            ).inc()
        elif "/authorize" in endpoint:
            http_requests_authorize.labels(
                method=request.method,
                status_code=str(response.status_code)
            ).inc()
        elif "/admin" in endpoint:
            http_requests_admin.labels(
                method=request.method,
                status_code=str(response.status_code)
            ).inc()
        requests_in_progress.labels(endpoint=request.path).dec()
        # Calc and store the request duration
        duration = time.time() - g.start_time
        request_duration.labels(
            endpoint=request.path,
            method=request.method
        ).observe(duration)
        return response

    return app


if __name__ == "__main__":
    # Cretaion of the Flask app
    flask_app = create_app()

    # Merge Flask with the /metrics (Prometheus) endpoint
    application = DispatcherMiddleware(flask_app, {
        "/metrics": make_wsgi_app()
    })

    print("OAuth2 server running on http://localhost:5001")
    print("Metrics available on http://localhost:5001/metrics")
    print("Endpoints:")
    print(" GET /authorize --Authorization Code flow (browser)")
```

```
    print(" POST /token --Token endpoint")
    print(" GET /admin/clients --List all clients")
    print(" POST /admin/clients --Register a new client")
    print(" DELETE /admin/clients/<id> --Delete a client")
    print()

    run_simple("0.0.0.0", 5001, application, use_reloader=True, use_debugger=True)
```

Los **decoradores** `@before_request` y `@after_request` son hooks de Flask que se ejecutan automáticamente antes y después de cada petición HTTP respectivamente, obligando a cada uno de las peticiones a pasar por ellos, haciendo que nuestras métricas recojan los datos, además de que nos permite registrar todas las peticiones en un único lugar sin modificar cada ruta individualmente, siguiendo el principio DRY (*Don't Repeat Yourself*).
Por otra parte el `DispatcherMiddleware` combina la aplicación Flask con la mini aplicación de Prometheus que genera el endpoint `/metrics`, de modo que todo el tráfico a `/metrics` lo gestiona Prometheus y el resto lo gestiona Flask.

6 Modificaciones a *app.py*

Se han añadido dos instrumentaciones importantes en `app.py` mediante decoradores Flask.

6.1 Exposición del endpoint /metrics

Se utiliza `DispatcherMiddleware` para combinar la aplicación Flask con la mini aplicación de Prometheus que genera el endpoint `/metrics`. Esto permite que Prometheus pueda leer las métricas en `http://localhost:5001/metrics`:

```
        from prometheus_client import make_wsgi_app
        from werkzeug.middleware.dispatcher import DispatcherMiddleware
        from werkzeug.serving import run_simple

        if __name__ == "__main__":
        flask_app = create_app()
        # Combine Flask with the Prometheus /metrics endpoint
        application = DispatcherMiddleware(flask_app, {
            "/metrics": make_wsgi_app()
        })
        run_simple("0.0.0.0", 5001, application,
        use_reloader=True, use_debugger=True)
```

Listing 6: Exposicion del endpoint /metrics

6.2 Registro automático de métricas HTTP

Hemos usado los decoradores `@before_request` y `@after_request` de Flask para registrar automáticamente todas las peticiones HTTP sin modificar cada ruta individualmente, para así seguir el patrón DRY (*Don't Repeat Yourself*):

```python
@app.before_request
def before_request():
# Store arrival time and increment in-progress counter
g.start_time = time.time()
requests_in_progress.labels(endpoint=request.path).inc()

@app.after_request
def after_request(response):
endpoint = request.path

# General HTTP counter
http_requests.labels(
method=request.method,
endpoint=endpoint,
status_code=str(response.status_code)
).inc()

# Specific counters per endpoint
if "/token" in endpoint:
http_requests_token.labels(
method=request.method,
status_code=str(response.status_code)
).inc()
elif "/authorize" in endpoint:
http_requests_authorize.labels(
method=request.method,
status_code=str(response.status_code)
).inc()
elif "/admin" in endpoint:
http_requests_admin.labels(
method=request.method,
status_code=str(response.status_code)
).inc()

# Decrement in-progress and record duration
requests_in_progress.labels(endpoint=request.path).dec()
duration = time.time() - g.start_time
request_duration.labels(
endpoint=request.path,
method=request.method
).observe(duration)

return response
```

Listing 7: Decoradores before y after request

6.3 Instrumentación de flujos OAuth2 — oauth2_routes.py

Hemos tenido que modificar este fichero puesto que nuestras métricas afectan a los 3 flujos de nuestra aplicación, siguiendo siempre la misma lógica se incrementa `started` al entrar en la función, `failed` en cada punto de fallo y `success` junto con los contadores de tokens al completarse con éxito.
Esto se entiende mejor con un ejemplo:

```
        def _handle_client_credentials():
        auth_flows_started.labels(
        grant_type="client_credentials").inc()

        client_id, err = _authenticate_client()
        if err:
        auth_flows_failed.labels(
        grant_type="client_credentials").inc()
        return _error_response("invalid_client", err, 401)

        client = get_client(client_id)
        if client is None or \
        "client_credentials" not in client["grant_types"]:
        auth_flows_failed.labels(
        grant_type="client_credentials").inc()
        return _error_response("unauthorized_client",
        "Client is not authorized for client_credentials")

        scope = request.form.get("scope", "")
        token_response = issue_token_pair(
        client_id=client_id,
        scope=scope,
        user_id=None,
        include_refresh=False,
        )

        auth_flows_success.labels(
        grant_type="client_credentials").inc()
        tokens_issued.labels(
        token_type="access",
        grant_type="client_credentials").inc()
        access_tokens_issued.labels(
        grant_type="client_credentials").inc()
        # client_credentials no emite refresh token (RFC 6749)

        return jsonify(token_response)
```

Listing 8: Fragmento de _handle_client_credentials()

De tal manera que el flujo de trabajo de la aplicación es el siguiente

`App OAuth2 (5001)` → `Prometheus (9090)` → `Grafana (3000)`

Ahora que ya tenemos los dos primeros pasos configurados, vamos a preparar el grafana.
Lo primero que hacemos es añadir Prometheus como fuente de datos en *Connections >Data Sources >Add data source*, se configura la URL `http://localhost:9090` y se verifica la conexión con *Save & Test*, obteniendo el mensaje:

"Successfully queried the Prometheus API."

7 Locust

Llegados a este punto ya tenemos toda la parte de configurado de la aplicación hecho, pero claro no hemos mencionado nada de Locust, esto se debe a que Locust simplemente va a realizar las pruebas de carga que nosotros dictemos sobre nuestro sistema, por lo que necesitábamos todo el trabajo previo para que esta aplicación pudiese funcionar de manera correcta.

7.1 Instalación

Locust se instala como una dependencia del proyecto mediante `uv`:

```
cd ~/oauth2
uv add locust
```

Se arranca con el siguiente comando, que lanza la interfaz web en el puerto 8089:

```
uv run locust -f locustfile.py --host=http://localhost:5001
```

```python
from locust import HttpUser, task, between


class OAuth2LoadTest(HttpUser):
    """
    Virtual Locust user for testing the OAuth2 server.

    This user simulates both legitimate traffic and controlled malicious
    traffic to observe the application behavior and its Prometheus metrics.

    Metrics covered:
        - auth_flows_started / auth_flows_success / auth_flows_failed
        - tokens_issued / access_tokens_issued / refresh_tokens_issued
        - http_requests / http_requests_token / http_requests_authorize /
          http_requests_admin
        - requests_in_progress / request_duration
        - active_tokens / tokens_revoked
        - pending_auth_codes
        - auth_code_reuse
        - revoked_token_usage
        - expired_token_usage (indirectly via invalid refresh token)
        - brute_force_attempts
        - user_auth_failed
        - active_clients

    Traffic distribution:
        Legitimate: 6+3+2 = 11/19 = 58% of traffic
        Malicious: 2+2+1+1+1 = 7/19 = 37% of traffic
        Monitoring: 1/19 = 5% of traffic
    """

    # Each virtual user waits between 1 and 3 seconds before the next task
    # This simulates real user think time between actions
    wait_time = between(1, 3)
```

```python

    # --Client credentials (M2M flow) ------------------------------------
    VALID_CLIENT_ID = "service-account"
    VALID_CLIENT_SECRET = "6Q6TZj_hKy4xBnZhgBuIwdJ6lSaQbF-9XWR9S_BHthk"

    # --Web application client (authorization_code flow) -----------------
    WEB_CLIENT_ID = "web-app"
    WEB_CLIENT_SECRET = "Vh_lxCQeixB4jAraZRxcd0kmP43YOeyXAn7IVomdMmc"
    WEB_REDIRECT_URI = "http://localhost:8080/callback"

    # --Valid test users defined in app.py -------------------------------
    VALID_USERNAME = "testuser"
    VALID_PASSWORD = "testpass"

    # ============================================================================
    # LEGITIMATE TRAFFIC
    # ============================================================================

    @task(6)
    def valid_client_credentials(self):
        """
        Legitimate M2M traffic: requests a token with correct credentials.
        This is the most frequent task because in production most traffic
        comes from machine-to-machine applications.

        Metrics fired:
            - auth_flows_started / auth_flows_success {grant_type="client_credentials"}
            - tokens_issued{token_type="access", grant_type="client_credentials"}
            - access_tokens_issued{grant_type="client_credentials"}
            - active_tokens{token_type="access"}
            - http_requests / http_requests_token with status 200
            - requests_in_progress / request_duration

        Probability: 32% (6 of 19)
        """
        response = self.client.post("/token", data={
            "grant_type": "client_credentials",
            "client_id": self.VALID_CLIENT_ID,
            "client_secret": self.VALID_CLIENT_SECRET,
            "scope": "read"
        })

        if response.status_code != 200:
            response.failure(f"Valid client_credentials failed with {response.status_code}"
                )

    @task(3)
    def valid_authorization_code_flow(self):
        """
        Complete authorization_code flow: login -> get code -> exchange for tokens.
        Without this task, pending_auth_codes would grow indefinitely during
        load tests because codes would be issued but never consumed.

```

```
        Metrics fired:
            - auth_flows_started / auth_flows_success {grant_type="authorization_code"}
            - tokens_issued{token_type="access"/"refresh", grant_type="authorization_code"}
            - access_tokens_issued / refresh_tokens_issued {grant_type="authorization_code
              "}
            - active_tokens{token_type="access"} / active_tokens{token_type="refresh"}
            - pending_auth_codes (incremented then decremented)
            - http_requests / http_requests_authorize / http_requests_token with status 200

        Probability: 16% (3 of 19)
        """
        # Step 1: POST /authorize with valid credentials -> get the auth code
        with self.client.post("/authorize", data={
            "client_id": self.WEB_CLIENT_ID,
            "redirect_uri": self.WEB_REDIRECT_URI,
            "response_type": "code",
            "scope": "read",
            "state": "locust-test",
            "action": "approve",
            "username": self.VALID_USERNAME,
            "password": self.VALID_PASSWORD
        }, catch_response=True, allow_redirects=False) as auth_response:

            # Server returns 302 with the code in the redirect URL
            if auth_response.status_code not in (200, 302):
                auth_response.failure(
                    f"Valid authorize failed with {auth_response.status_code}"
                )
                return

            # Extract the code from the Location header
            code = None
            location = auth_response.headers.get("Location", "")
            if "code=" in location:
                code = location.split("code=")[1].split("&")[0]

            if not code:
                auth_response.success()
                return

            auth_response.success()

        # Step 2: POST /token exchanging the code -> get access + refresh token
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
        }, catch_response=True) as token_response:

            if token_response.status_code != 200:
                token_response.failure(
```

```
                f"Code exchange failed with {token_response.status_code}"
            )
            return

        token_response.success()

    @task(2)
    def valid_refresh_token_flow(self):
        """
        Complete refresh_token flow: get tokens via authorization_code,
        then renew them using the refresh token.
        Without this task, tokens_revoked and refresh_tokens_issued
        would always be zero during load tests.

        Metrics fired:
            - auth_flows_started / auth_flows_success {grant_type="refresh_token"}
            - tokens_issued / access_tokens_issued / refresh_tokens_issued
            - active_tokens (new tokens activated)
            - tokens_revoked (old refresh token revoked via token rotation)
            - http_requests_token with status 200

        Probability: 11% (2 of 19)
        """
        # Step 1: Get a valid pair of tokens via authorization_code
        code = self._get_authorization_code()
        if not code:
            return

        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
        }, catch_response=True) as token_response:

            if token_response.status_code != 200:
                token_response.failure(
                    f"Token acquisition for refresh failed with {token_response.status_code}
                        "
                )
                return

            try:
                tokens = token_response.json()
                refresh_token = tokens.get("refresh_token")
            except Exception:
                token_response.failure("Could not parse token response")
                return

            token_response.success()

        if not refresh_token:
```

```
            return

        # Step 2: Renew tokens using the refresh token
        with self.client.post("/token", data={
            "grant_type": "refresh_token",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "refresh_token": refresh_token
        }, catch_response=True) as refresh_response:

            if refresh_response.status_code != 200:
                refresh_response.failure(
                    f"Valid refresh_token failed with {refresh_response.status_code}"
                )
                return

            refresh_response.success()

    # ============================================================================
    # MALICIOUS TRAFFIC / ATTACK SIMULATION
    # ============================================================================

    @task(2)
    def brute_force_api(self):
        """
        M2M brute force attack: tries to guess the client secret.
        The server should defend correctly returning 401.

        Metrics fired:
            - auth_flows_started / auth_flows_failed {grant_type="client_credentials"}
            - brute_force_attempts{client_id="service-account"}
            - http_requests / http_requests_token with status 401

        Probability: 11% (2 of 19)
        """
        with self.client.post("/token", data={
            "grant_type": "client_credentials",
            "client_id": self.VALID_CLIENT_ID,
            "client_secret": "wrong_secret_attempt"
        }, catch_response=True) as response:

            # 401 is the expected behavior, server defended correctly
            if response.status_code == 401:
                response.success()
            else:
                response.failure(f"Expected 401 but got {response.status_code}")

    @task(2)
    def brute_force_user_login(self):
        """
        Web brute force attack: tries to guess a user password.
        The action field is required for the app to reach the credential
        check. Without it the app fails before checking the password
```

```
        and user_auth_failed would never be incremented.

        Metrics fired:
            - user_auth_failed{username=..., client_id="web-app"}
            - http_requests / http_requests_authorize with status 401

        Probability: 11% (2 of 19)
        """
        with self.client.post("/authorize", data={
            "client_id": self.WEB_CLIENT_ID,
            "redirect_uri": self.WEB_REDIRECT_URI,
            "response_type": "code",
            "scope": "read",
            "state": "locust-test",
            "action": "approve", # required for app to reach login check
            "username": self.VALID_USERNAME,
            "password": "wrong_password_attempt"
        }, catch_response=True) as response:

            if response.status_code == 401:
                response.success()
            else:
                response.failure(f"Expected 401 but got {response.status_code}")

    @task(1)
    def auth_code_replay_attack(self):
        """
        Authorization code replay attack: tries to use the same auth code twice.
        The first exchange should succeed (200), the second should fail (400)
        because the code is already marked as used in the database.
        This is more realistic than using an access token as refresh token.

        Metrics fired:
            - pending_auth_codes (incremented then decremented on first exchange)
            - auth_code_reuse (incremented on second exchange attempt)
            - http_requests_token with status 200 then 400

        Probability: 5% (1 of 19)
        """
        # Get a valid auth code
        code = self._get_authorization_code()
        if not code:
            return

        # First exchange: should succeed (200)
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
        }, catch_response=True) as first_response:

```

```
            if first_response.status_code != 200:
                first_response.failure(
                    f"First exchange failed unexpectedly with {first_response.status_code}"
                )
                return

            first_response.success()

        # Second exchange of the same code: should fail (400)
        # This fires auth_code_reuse in models.py -> consume_authorization_code()
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
        }, catch_response=True) as second_response:

            if second_response.status_code in (400, 401):
                second_response.success()
            else:
                second_response.failure(
                    f"Expected 400/401 on code replay but got {second_response.status_code}"
                )

    @task(1)
    def revoked_token_replay_attack(self):
        """
        Token replay attack: gets a valid refresh token, rotates it
        (which revokes the old one) and then tries to use the revoked token.
        This is more realistic than using an access token as refresh token
        because it simulates a real token theft scenario.

        Metrics fired:
            - tokens_revoked{token_type="refresh"} / tokens_revoked{token_type="access"}
            - active_tokens (decremented on revocation)
            - revoked_token_usage (on attempt to use the revoked token)
            - http_requests_token with status 400

        Probability: 5% (1 of 19)
        """
        # Step 1: Get valid tokens
        code = self._get_authorization_code()
        if not code:
            return

        refresh_token = None
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
```

```
        }, catch_response=True) as token_response:

            if token_response.status_code != 200:
                token_response.failure(
                    f"Token acquisition failed with {token_response.status_code}"
                )
                return

            try:
                tokens = token_response.json()
                refresh_token = tokens.get("refresh_token")
            except Exception:
                token_response.failure("Could not parse token response")
                return

            token_response.success()

        if not refresh_token:
            return

        # Step 2: Rotate the token (this revokes the old refresh token)
        with self.client.post("/token", data={
            "grant_type": "refresh_token",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "refresh_token": refresh_token
        }, catch_response=True) as rotate_response:

            if rotate_response.status_code != 200:
                rotate_response.success()
                return

            rotate_response.success()

        # Step 3: Try to use the ORIGINAL refresh token (now revoked)
        # This fires revoked_token_usage in models.py -> validate_refresh_token()
        with self.client.post("/token", data={
            "grant_type": "refresh_token",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "refresh_token": refresh_token
        }, catch_response=True) as revoked_response:

            if revoked_response.status_code in (400, 401):
                revoked_response.success()
            else:
                revoked_response.failure(
                    f"Expected 400/401 on revoked token but got {revoked_response.
                        status_code}"
                )

    @task(1)
    def invalid_refresh_token(self):
```

```
        """
        Tries to use a completely fake refresh token.
        Tests the expired/invalid token error path.

        Metrics fired:
            - auth_flows_started / auth_flows_failed {grant_type="refresh_token"}
            - http_requests / http_requests_token with status 400

        Probability: 5% (1 of 19)
        """
        with self.client.post("/token", data={
            "grant_type": "refresh_token",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "refresh_token": "completely_fake_refresh_token_xyz123"
        }, catch_response=True) as response:

            if response.status_code == 400:
                response.success()
            else:
                response.failure(
                    f"Expected 400 for fake refresh token but got {response.status_code}"
                )

    # ============================================================================
    # MONITORING ENDPOINTS
    # ============================================================================

    @task(1)
    def list_admin_clients(self):
        """
        Queries the list of registered clients.
        Generates traffic on the admin endpoint metrics.

        Metrics fired:
            - http_requests / http_requests_admin with status 200
            - requests_in_progress / request_duration for /admin/clients

        Probability: 5% (1 of 19)
        """
        response = self.client.get("/admin/clients")

        if response.status_code != 200:
            response.failure(f"/admin/clients GET failed with {response.status_code}")

    # ============================================================================
    # PRIVATE HELPER METHOD
    # ============================================================================

    def _get_authorization_code(self) -> str | None:
        """
        Performs the authorization step (POST /authorize) with valid credentials
        and returns the auth code extracted from the Location header.
```

```
        Returns None if the code could not be obtained.

        This helper is used by multiple tasks to avoid repeating the login
        logic in each one (DRY principle).

        Note: the action field is required for the app to reach the
        credential check. Without it the app fails before authenticating
        the user and no auth code is issued.
        """
        with self.client.post("/authorize", data={
            "client_id": self.WEB_CLIENT_ID,
            "redirect_uri": self.WEB_REDIRECT_URI,
            "response_type": "code",
            "scope": "read",
            "state": "locust-internal",
            "action": "approve", # required field
            "username": self.VALID_USERNAME,
            "password": self.VALID_PASSWORD
        }, catch_response=True, allow_redirects=False) as response:

            if response.status_code not in (200, 302):
                response.failure(
                    f"_get_authorization_code failed with {response.status_code}"
                )
                return None

            # Extract the code from the Location header (302 redirect)
            code = None
            location = response.headers.get("Location", "")
            if "code=" in location:
                code = location.split("code=")[1].split("&")[0]

            response.success()
            return code


    @task(1)
    def client_enumeration_attack(self):
        """
        Client enumeration attack: tries to access with unknown client_ids.
        Fires: unknown_client_attempts
        Probability: ~5% (1 of 20)
        """
        import random
        fake_clients = [
            "admin", "root", "test", "app", "api",
            "mobile", "backend", "service", "user", "guest"
        ]
        fake_client = random.choice(fake_clients)

        with self.client.post("/token", data={
            "grant_type": "client_credentials",
            "client_id": fake_client,
```

```
            "client_secret": "any_secret"
        }, catch_response=True) as response:
            if response.status_code == 401:
                response.success()
            else:
                response.failure(f"Expected 401 but got {response.status_code}")
    @task(1)
    def credential_stuffing_attack(self):
        """
        Credential stuffing attack: tries many different usernames
        with common passwords. Different from brute force which
        attacks one user with many passwords.
        Fires: user_auth_failed, unique_failed_usernames
        Probability: ~5% (1 of 20)
        """
        import random
        # Simulates a list of leaked credentials from other sites
        leaked_credentials = [
            ("john.doe", "password123"),
            ("jane.smith", "qwerty"),
            ("carlos.garcia", "123456"),
            ("maria.lopez", "letmein"),
            ("pedro.martinez", "welcome"),
            ("ana.rodriguez", "monkey"),
            ("luis.hernandez", "dragon"),
            ("sofia.gonzalez", "master"),
        ]
        username, password = random.choice(leaked_credentials)

        with self.client.post("/authorize", data={
            "client_id": self.WEB_CLIENT_ID,
            "redirect_uri": self.WEB_REDIRECT_URI,
            "response_type": "code",
            "scope": "read",
            "state": "locust-test",
            "action": "approve",
            "username": username,
            "password": password
        }, catch_response=True) as response:
            if response.status_code == 401:
                response.success()
            else:
                response.failure(f"Expected 401 but got {response.status_code}")

    @task(1)
    def token_harvesting_attack(self):
        """
        Token harvesting: requests valid tokens but never uses or
        revokes them. active_tokens gauge will keep growing.
        Fires: active_tokens (keeps incrementing without decrementing)
        Probability: ~5% (1 of 20)
        """
        # Just request tokens without ever refreshing or revoking them
```

```
560         # This simulates an attacker accumulating valid tokens
561         with self.client.post("/token", data={
562             "grant_type": "client_credentials",
563             "client_id": self.VALID_CLIENT_ID,
564             "client_secret": self.VALID_CLIENT_SECRET,
565             "scope": "read"
566         }, catch_response=True) as response:
567             if response.status_code == 200:
568                 # Intentionally do nothing with the token
569                 # In a real attack the attacker would store it
570                 response.success()
571             else:
572                 response.failure(f"Expected 200 but got {response.status_code}")
```

7.2 Explicación del diseño del locustfile

7.3 Configuración de las pruebas

Desde la interfaz web de Locust vamos a configurar dos tipos de pruebas:

Prueba de carga — Evalúa el comportamiento del sistema en condiciones normales:

Parámetro	Valor
Number of users	50
Spawn rate	5 usuarios/segundo
Duración	5 minutos

Cuadro 2: Configuración de la prueba de carga

Prueba de estrés — Evalúa el comportamiento del sistema en condiciones extremas para encontrar el límite real:

Parámetro	Valor
Number of users	200
Spawn rate	10 usuarios/segundo
Duración	3 minutos

Cuadro 3: Configuración de la prueba de estrés

7.4 Protección de la VM durante las pruebas

Debido a los recursos de nuestra VM, nos vemos con un problema similar al de la práctica 1, no podemos lanzar peticiones como locos, sin preocuparnos de las consecuencias para nuestro sistema, por ello hemos decidido tomar una ciertas precauciones por lo que arrancamos prometheus con unos límites explícitos de retención de datos:



Figura 1: Como podemos observar con los parámetros del locust del cuadro 2 obtenemos una resultados normales, es decir el sistema puede manejar la carga de trabajo de manera normal

```
./prometheus --config.file=prometheus.yml \
--storage.tsdb.retention.time=2h \
--storage.tsdb.retention.size=500MB
```

Si el otro *dashboard* de grafana nos indica que estamos muy cerca de los límites, podemos llegar a lanzar estos comandos para tener un control a tiempo real sin necesidad de estar cambiando de panel constantemente:

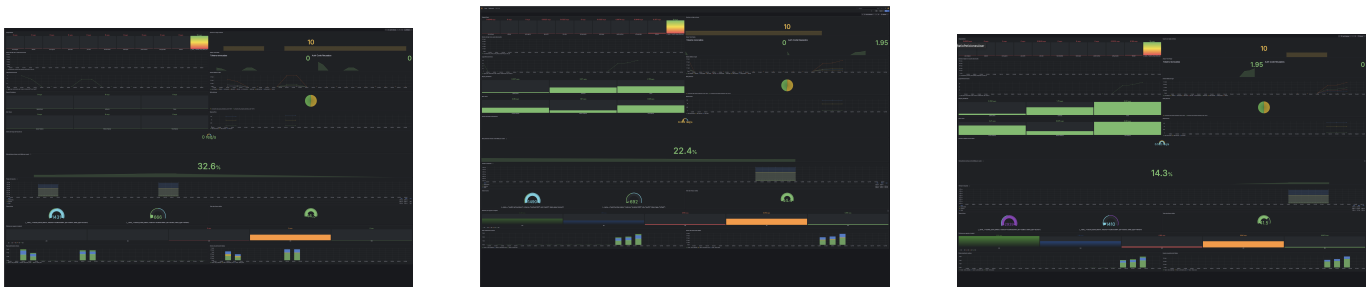
```
watch -n 2 'echo "=== DISCO ===" && df -h / && \
echo "=== RAM ===" && free -h && \
echo "=== CPU ===" && top -bn1 | head -5'
```

7.5 Comportamientos del sistema frente a los dos escenarios de prueba

En este apartado mostraremos una captura general del grafana en el que podremos comprobar cómo se ha comportado nuestro sistema ante las prubeas que hemos considerado adecuadas para sus dos escenarios.
Explicaremos en detalle cada uno de los paneles más adelante en este informe pero es una representación visual de cómo se está comportando el sistema

7.5.1. Condiciones normales

7.5.2. Condiciones de estrés



(a) 200 ususarios totales. (b) 150 usuarios totales. (c) 125 usuarios totales.

Figura 2: Pruebas de carga con diferentes usuarios totales

Si nos fijamos hemos hecho pruebas de estres con diferentes números de usuarios totales la cuestión es que manteniendo constantes los números de usuarios concurrentes por segundo, nuestra vm no puede con ellos, debido a las limitaciones de memoria RAM y de CPU, con esto demostramos que tenemos un cuello de botella en la concurrencia, por lo que modificamos el número de usuarios concurrentes para que la prueba de estrés se acerque al límite del sistema, tomaremos como referente 150 usuarios, ya que es un número realista, pero reduciremos el número de usuarios concurrentes para que nuestro sistema pueda atender dichas peticiones.

7.5.3. Flujo de ejecición de las aplicaciones

1. Terminal 1 `uv run python app.py`
2. Terminal 2 `./prometheus -config.file=prometheus.yml`
3. Terminal 3 `uv run python test.py`
   `uv run locust -f locustfile.py -host=http://localhost:5001`
4. Terminal 4 (opcional, sólo si tenemos pocos recursos en la VM)`watch` para vigilancia de recursos

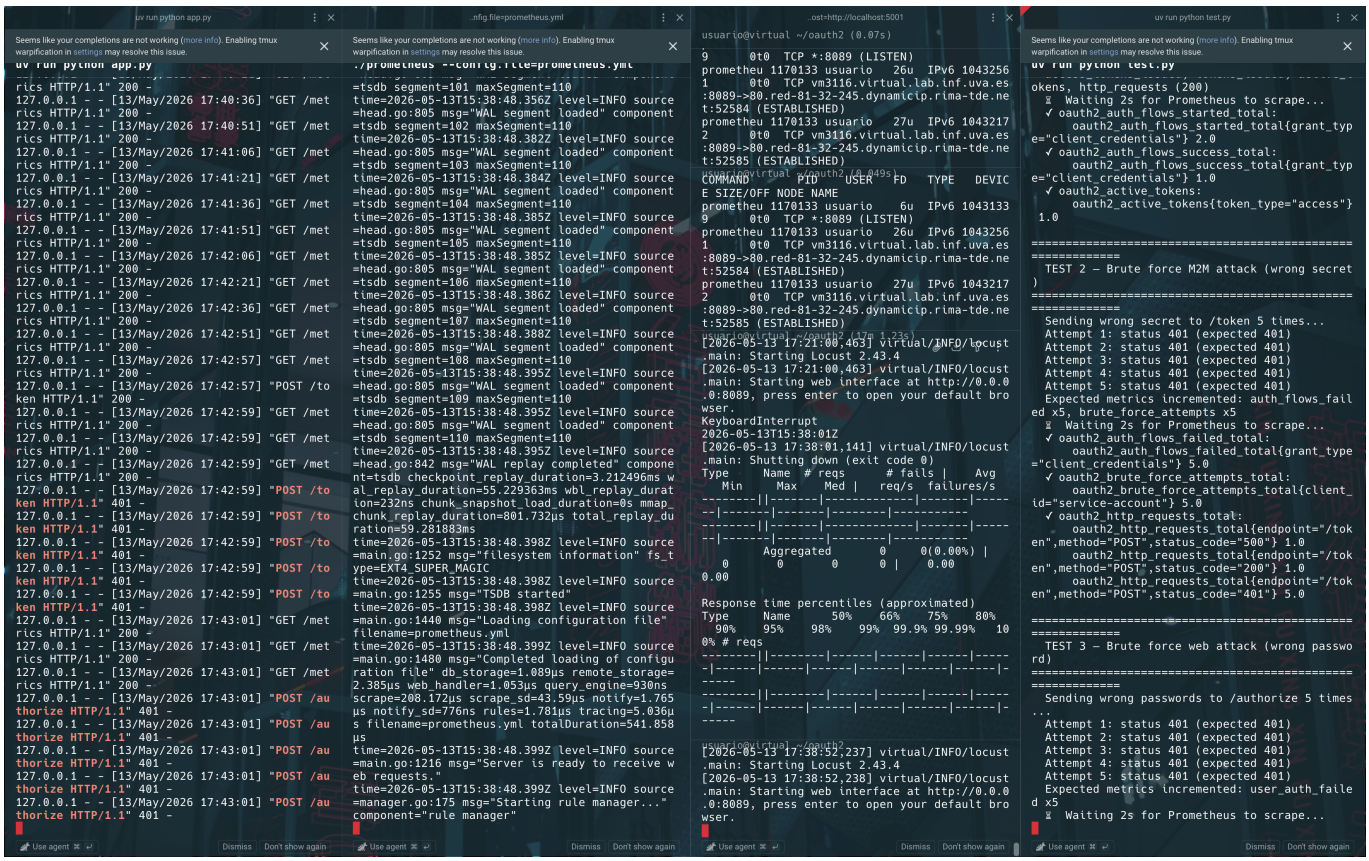


Figura 3: Ejecutamos la *app.py*, el locust y el prometheus, con el error de ejecución en los puertos erróneos

Y el orden de parada es el inverso, esperando 30 segundos entre Locust y Prometheus para que este último recoja los últimos datos antes de apagarse.

8 Verificación manual de métricas EDR

En el punto anterior, hemos mencionado el `test.py`, el cual es un script que comprueba cada alguna métrica de manera individual, en concreto comprobamos las siguientes:

Test	Nombre	Métricas verificadas
1	client_credentials válido	auth_flows_success, active_tokens
2	Fuerza bruta M2M	auth_flows_failed, brute_force_attempts
3	Fuerza bruta web	user_auth_failed
4	authorization_code completo	refresh_tokens_issued, pending_auth_codes
5	refresh_token válido	tokens_revoked, active_tokens
6	Token replay attack	revoked_token_usage
7	Auth code reuse	auth_code_reuse
8	Peticiones en paralelo	requests_in_progress, request_duration
9	Token falso/expirado	expired_token_usage

Cuadro 4: Tests del script de verificación manual

9 Enfoque EDR del sistema de métricas

El enunciado de la práctica señala explícitamente que uno de los objetivos de las pruebas de carga es identificar *"si surgen fallos de seguridad fruto del alto nivel de carga"* y que *"se podrán añadir métricas adicionales"*. Partiendo de este punto, hemos optado por un enfoque orientado a la ciberseguridad, intentando emular un EDR (*Endpoint Detection and Response*), que permite detectar comportamientos anómalos en tiempo real durante las pruebas de carga.

Un sistema EDR monitoriza eventos de seguridad en tiempo real para detectar patrones de ataque como fuerza bruta, reutilización de tokens o intentos de acceso con credenciales comprometidas. Prometheus y Grafana permiten replicar este comportamiento de forma sencilla sobre el servidor OAuth2.

9.1 Métricas adicionales implementadas

Se han añadido tres categorías de métricas adicionales sobre las requeridas por el enunciado:

9.1.1. Rendimiento del sistema

Métrica	Descripción
`oauth2_request_duration_seconds`	Histogram del tiempo de respuesta por endpoint y método HTTP. Permite calcular el percentil 95 que el enunciado fija en 250ms como umbral de referencia.
`oauth2_requests_in_progress`	Gauge de peticiones HTTP simultáneas en curso en este momento. Útil para detectar ataques DoS bajo pruebas de estrés.

Cuadro 5: Métricas de rendimiento

9.1.2. Estado de tokens

Métrica	Descripción
oauth2_active_tokens	Gauge de tokens activos en el sistema por tipo. Sube al emitir tokens y baja al revocarlos.
oauth2_tokens_revoked_total	Counter de tokens revocados por tipo. Refleja la rotación de tokens en el flujo refresh_token.
oauth2_pending_auth_codes	Gauge de códigos de autorización emitidos pero no canjeados todavía. Un valor creciente indica clientes que piden códigos sin completar el flujo.
oauth2_revoked_token_usage_total	Counter de intentos de uso de tokens ya revocados. Señal de posible token replay attack.
oauth2_expired_token_usage_total	Counter de intentos de uso de tokens expirados.
oauth2_auth_code_reuse_total	Counter de intentos de reutilización de códigos de autorización ya consumidos. Señal de authorization code replay attack.

Cuadro 6: Métricas de estado de tokens

9.1.3. Detección de ataques

Métrica	Descripción
oauth2_brute_force_attempts_total	Counter de intentos fallidos de autenticación por client_id. Detecta ataques de fuerza bruta M2M.
oauth2_user_auth_failed_total	Counter de intentos fallidos de login por username y client_id. Detecta ataques de fuerza bruta sobre usuarios.

Cuadro 7: Métricas de detección de ataques

10 Paneles de grafana

Al igual que en la anterior entrega, hemos tenido que hacer un *dashboard* en grafana para tener una reperesentación gráfica de nuestro trabajo en esta práctica



Figura 4: Nuestro panel de grafana para esta práctica. Aunque entraremos más en detalle, nuestro *dashboard* presenta dos tipos de metricas, las clásicas de pruebas de carga y las métricas específicas para nuestro enfoque EDR

Una vez que ya tenemos una idea de cómo hemos ditribuido el *dashboard* vamos a ir panel a panel explicándo un poco la intención y alguna peculiaridad de la representación.
Las consultas con Prometheus son mucho más simples que las que nos tocó hacer con Postgres en la práctica anterior, pero al ser una herramienta nueva, nos ha llevado nuestro tiempo acostumbrando a su sintaxis y sus operandos, teniendo que pedir consejo a Adrián sobre como escribir las peticiones para poder reflejar de una manera adecuada los datos.

10.1 Ataques de fuerza burta vs *credential stuffing*

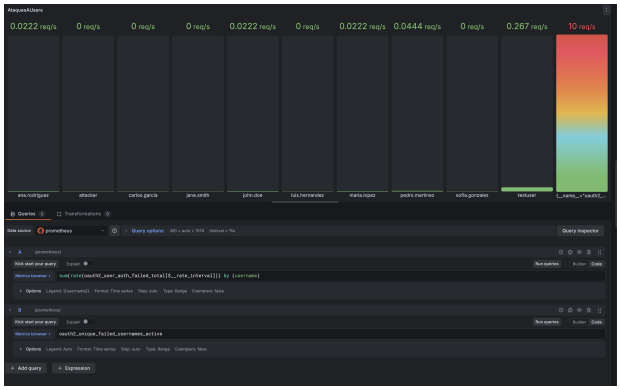


Figura 5: Con este panel estamos intentando diferenciar entre los errores de un usuario al escribir su contraseña con un posible ataque de fuerza bruta para entrar a la cuenta de un usuario.

Esta métrica pertenece a nuestro enfoque EDR, pero para entenderla bien, vamos a explicar cuáles son los dos tipos de ataques que estamos midiendo. Un *Brute Forze Attack* trata de una técnica de hacking que se basa en probar el acceso a la cuenta de un usuario empleando un diccionario de contraseña, mientras que el *credential stuffing* se basa en intentar *loggearse* en un sistema empleando contraseñas de otros sitios de ese mismo usuario/mail.
Este panel muestra qué usuarios han sido atacados durante las pruebas, aquellos con más intentos fallidos son víctimas de fuerza bruta mientras que la presencia de muchos usuarios distintos con pocos intentos cada uno indica *credential stuffing*.

10.2 Ratio de Tokens

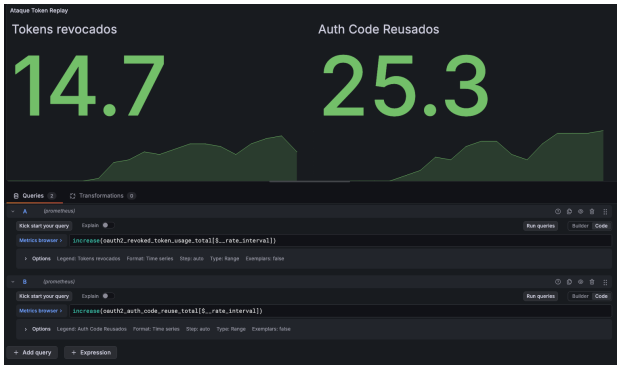


Figura 6: Tipos de tokens que maneja nuestro sistema

Con este panel lo que estamos intentando mirar es si estamos sufriendo ataques del tipo *Token Replay*, los cuales se basan en que un atacante ha conseguido un token válido y lo está empleando desde su cuenta para poder realizar ataques sobre nuestro sistema.
Para tener una visión limpia sobre este tipo de ataques estamos usando las fuunciones de incremento absoluto (*increase()*) en PromQL, las cuales revelan el número exacto de intentos de ataque neutralizados durante la ventana de evaluación. Además empleamos la métrica `oauth2_revoked_token_usage_total` para contabilizar las peticiones bloqueadas que intentan operar utilizando un *Refresh Token* que ya ha sido invalidado. ya que cualquier intento posterior de usarlas no solo es denegado, sino que queda registrado en este contador como una brecha de seguridad interceptada.

10.3 Fallos del login de los usuarios



Figura 7: Número de usuarios de nuestro sistema que han tenido fallos en su login

A simple vista, esta métrica parece un poco irrelevante ¿qué información útil aporta el número de usuarios que se han equivocado al introducir sus datos al hacer el *login*?, y la respuesta a esta pregunta es simple, un alto número de usuarios que fallan en su inicio de sesión es un indicativo de que algo anda mal, de tal manera que unido al panel anterior, nos da la información de que puede que uno de esos usuarios esté sufiendo un ataque y que por ende sea un vector de infección a nuestro sistema.

10.4 Fallos del login de los usuarios



Figura 8: Usuarios de nuestro sistema que han tenido fallos en su login

Métrica complementaria a la anterior, la cuál ya no sólo nos dice los intentos fallidos de *login en el sistema*, sino que tenemos una visión de qué usuarios están teniendo este comportamiento.
Decimos que es complementaria puesto que si el adminitrador del sistema detecta en el panel anterior un número grande de errores de login por parte de usuarios, consultado este panel puede ver qué usuario en concreto está teniendo este comportamiento, lo que permite contactar con él para que revise la actividad o en su defecto, cortarle el acceso al sistema por si te trata de un bot o un usuario que ha perdido el acceso a sus credenciales.
Así como dato extra a gráfica se construye mediante una consulta PromQL que calcula la tasa de fallos (*rate()*) y los agrupa por nombre de usuario (sum(...) by (username)), para poder realizar esta distinción.

10.5 Logins correctos en el sistema



Figura 9: Número de logins correctos en el sistema

Esta métrica es una clásica en lo que se refiere al monitoreo de un sistema y a las pruebas de carga, con locust estamos testeando una carga de usuarios y peticiones simultáneas de tal manera que este panel nos sirve para ver cuántos usuarios están intentando acceder a nuestro sistema.

10.6 Peticiones a los diferentes endpoints de nuestro sistema

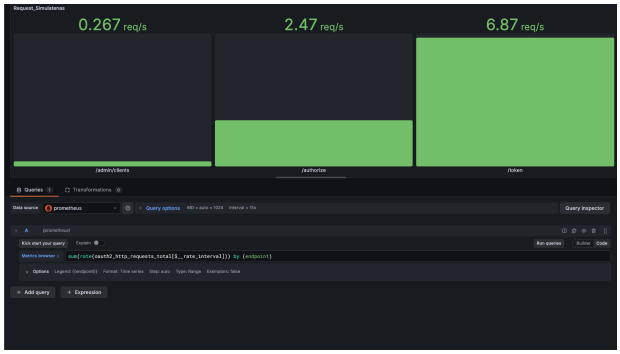


Figura 10: Peticiones que le llegan a los diferentes endpoints al sistema.

En este panel, nos encontramos con la distribución de peticiones a nuestro sistema.
Nos puede llamar la atención la distribución tan dispar de peticiones a cada uno de los *endpoints* pero hay que tener en cuenta que (`authorize`) es el endpoint que refleja una interacción humana, es decir, sería el formulario que seguiría un cliente al autenticarte en el sistema, por lo que, siguiendo el consejo de Adrián hemos generado una carga de trabajo para ello.
Sin embargo el `/admin/clients` es la ruta de administración interna, la cual no debería estar expuesto a ataques del exterior, por lo que registra un tráfico residual y estable.
Hemos decidido implementar este panel para tener una información de la carga de trabajo en nuestro sistema, ya que podemos modificar los recursos destinados a cada uno o en caso de tener que tomar medidas saber exactamente la parte de nuestro sistema que está lastrando la *performance* del entorno.

10.7 Distribución de las peticiones en nuestro sistema

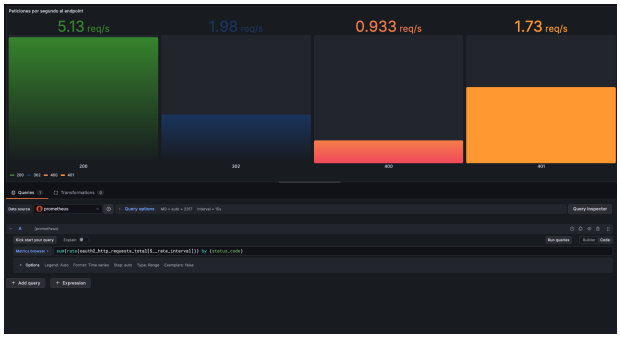


Figura 11: Distribución de las peticiones en nuestro sistema.

Panel que sirve para ver el estado de sistemas, el 200 es el código para las peticiones correcta por eso se representa de color ver mientras que las peticiones en el sector de color amarillo agrupan todas las respuestas que no han resultado en un éxito, principalmente errores 400 de *Bad Request*, 401 de *Unauthorized* y posibles redirecciones 302., los cuales nos aportan una información muy relevante ya que los errores pueden implicar un mal funcionamiento del sistema.

10.8 Ratio peticiones correctas fallidas por los usuarios



Figura 12: Tipos de tokens que maneja nuestro sistema

Nuestra idea con este panel es ver cómo se comportaba nuestro sistema ante los flujos de autenticación, como ya hemos comentado, nuestras pruebas no sólo buscan una prueba de carga sino comproabr la seguridad del sistema por lo que esté gráfico nos dice que el sistema es capaz de mantener un fijo constante de autorizaciones válidas pese a todo el ruido de peticiones malicosas que le estamos haciendo, teniendo un comprotamiento estable por lo que se puede ver en la linea de tendencias y que además, no está dando falsos positivos, es decir estamos ante un sistema robusto en lo referente a los *logeos*.

10.8.1. Tiempo de respuesta a cada *endpint*

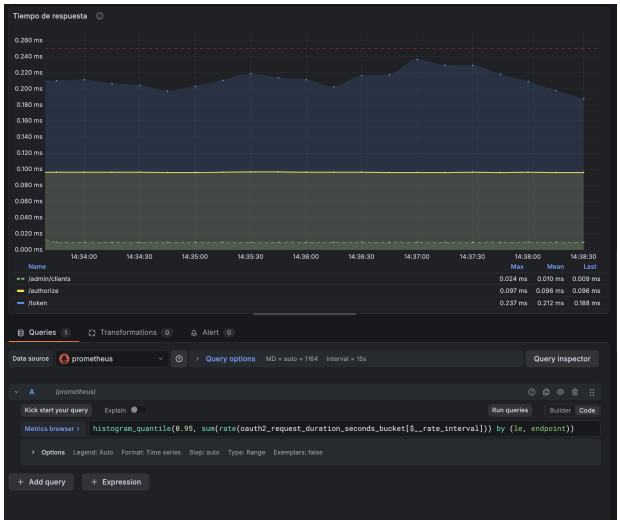


Figura 13: Gráfico que mietras el tiempo de respuesta a cada endpoint con una leyenda la cual nos da información exacta sobre los valores límite

Esta gráfica es de vita importancia para saber si los *endpoints* están funcionando como deberían, además de darnos la información de los tiempos de respuesta en función de la carga de trabajo que nosotros le estamos haciendo con el locuts.
Tenemos los 3 puntos , cada uno de un color, *token* (línea azul) es el punto de máximo trabajo de nuestra aplicación puesto que se encarga de la gestión de token y los registros de audotia, *authorized* (línea amarilla) es en el que se reflejan la respuesta de los usuarios por lo que la respuesta es casi instantánea mientras que el de *admin* al ser un *endpoint* interno tiene un tiempo de respues mínimo ya que no está expuesto a la interacción con los usuarios.

10.8.2. Tipos de *tokens* en nuestro sistema.



Figura 14: Información sobre todos los *tokens* emitidos en nuestro sistema.

Con este panel estamos registrando los tokens que se están emitiendo/empleando en nuestro sistema, debemos recordar que estamos empleando un servidor oauth2 para la autenticación por lo que esta estadistica es necesaria para comprobar que el sistema funciona bien y que no estamos sufriendo ningún tipo de ataque.
Como tenemos 3 tipos vamos a explicar cada uno de ellos

- **Query A (*Access Tokens*):** representa el número de sesiones activas que pueden realizar peticiones a recursos protegidos.
- **Query B (*Refresh Tokens*):** representa el número de sesiones de larga duración que permiten a los clientes obtener nuevas credenciales sin intervención del usuario.
- **Query C (*Total Tokens*):** como su nombre indica la suma de las anteriores.

10.8.3. Distribución de *tokens* en el sistema

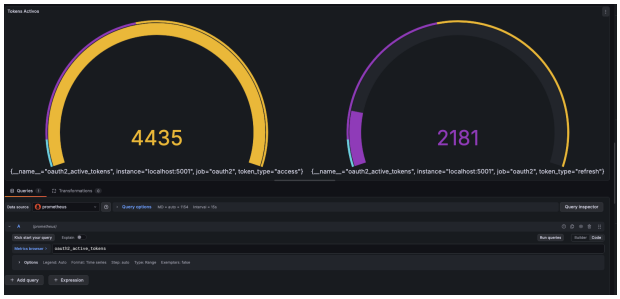


Figura 15: Información sobre todos los *tokens* emitidos en nuestro sistema.

Esta métrica es un histórico de todos los *tokens* emitidos en el sistema a nivel de acumulador, si nos damos cuanta esta métrica y la anterior tienen una relación, ya que una nos sirve de acumulador mientras que la anterior nos da una información de la producción de token y los tipos de cada uno.

10.8.4. Intentos de ataque de fuerza bruta

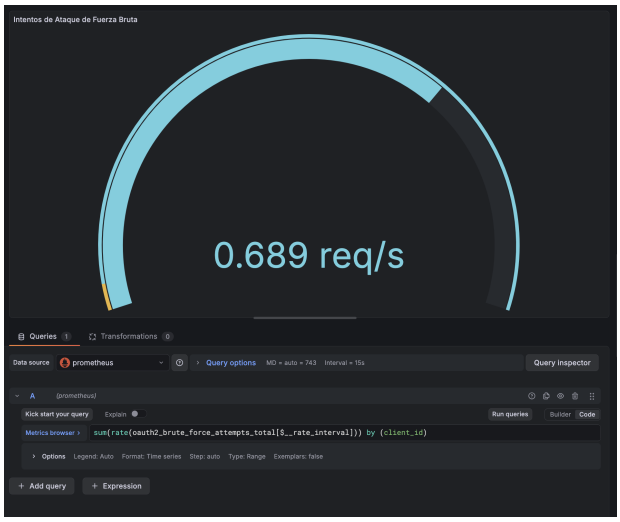


Figura 16: Peticiones consideradas de fuerza bruta que está recibiendo el sistema.

Esta métrica es vital para el enfoque EDR, puesto que nos da una información sobre un posible ataque al sistema. Nuestro enfoque ha sido el siguiente, tenemos una métrica que se incrementa cuando un usuario falla el login, debido a las condiciones del sistema no podemos tener un indicador más grande a puro código, pero si nos plantamos en el grafana, tenemos un panel para los intentos fallidos de login por usuario, lo que unido a un valor elevado de las *requests per second* en este panel nos permite aislar a aqellos usuarios que son probelmáticos.

10.8.5. Flujos de autenticació en cada uno de los *endpoints*

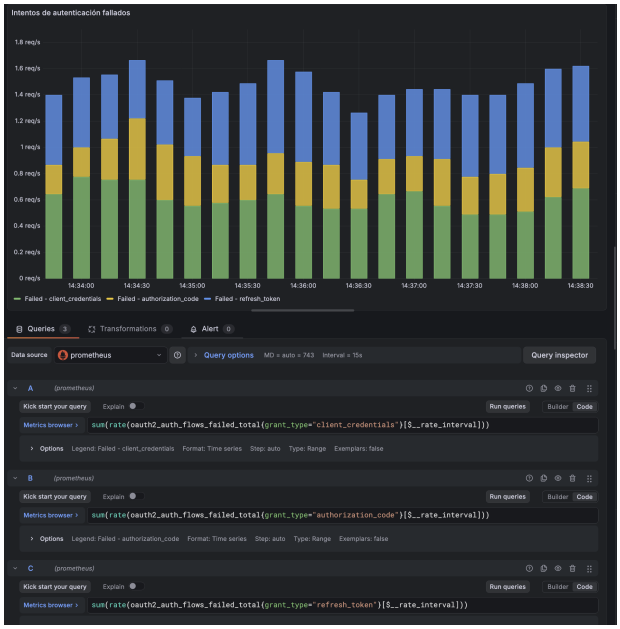


Figura 17: Tipos de tokens que maneja nuestro sistema

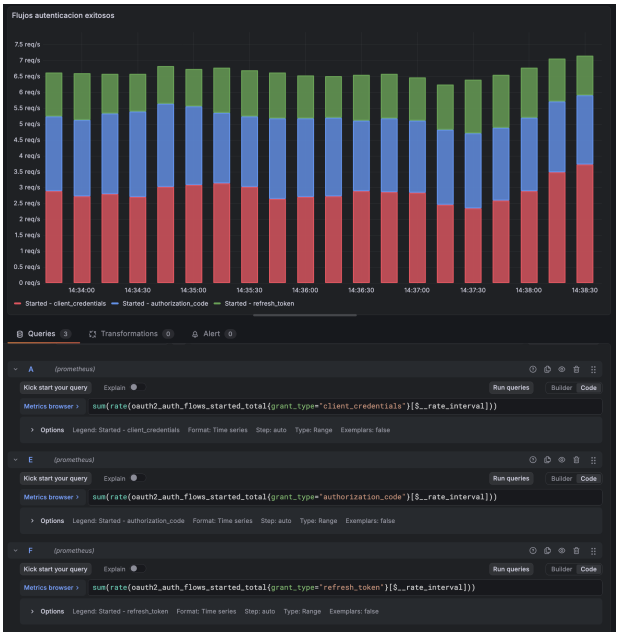


Figura 18: Tipos de tokens que maneja nuestro sistema

Para este panel lo que hemos buscado tener una representación visual de los intentos fallidos de inicio de sesión en cada uno de los *endpoints de nuestro sistema*

- **Verde**: registra fallos en la autenticación máquina-a-máquina. Una tasa alta aquí indica intentos de fuerza bruta contra secretos de aplicaciones o cuentas de servicio.
- **Amarillo**:representa intentos fallidos de inicio de sesión en el formulario web, usualmente vinculados a ataques de diccionario o *credential stuffing*.
- **Azul:** coontabiliza intentos de usar tokens de refresco inválidos, caducados o revocados. Es un indicador clave de ataques de *Replay* o de sesiones persistentes comprometidas.

10.9 Intento de login por parte de usuarios desconocidos

Figura 19: Peticiones de inicio de sesión de usuarios desconocidos para nuestro sistema.

Esta métrica es interesante, ya que estamos empleando un servidor de autenticación con una bbdd para registrar los usuarios con acceso a nuestros recursos, en caso de detectar algún incremento en esta métrica, podríamos deducir que el sistema se ha visto comprometido, adicionalmente casa muy bien con el enfoque que hemos decidido darle a la práctica ya que buscamos detectar posibles fallas cuando la carga de trabajo del sistema es muy grande, por lo que detectaríamos cualquier fallo en dicho panel. Aunque sí que es cierto que si revisamos el *app.py* el sistema está planteado para evitar este tipo de peticiones pero mejor prevenir que curar.